Rising Waters: Machine Learning Flood Prediction System

Project Description:

Rising Waters is a production-grade, web-based Machine Learning application designed to predict the likelihood of floods using historical weather and rainfall data. The system provides users with real-time flood risk estimates by analyzing critical environmental parameters such as temperature, humidity, cloud cover, and seasonal rainfall distributions.Unlike simplified prediction tools, Rising Waters incorporates a comprehensive backend architecture. It features secure user authentication, allowing users to maintain a personalized, database-backed history of their predictions. The core intelligence is powered by an XGBoost classification model, trained on data to ensure high accuracy despite the rarity of flood events. Built with Flask, PostgreSQL, and SQLAlchemy, the application processes user inputs, dynamically engineers features, and delivers instant, reliable risk assessments.

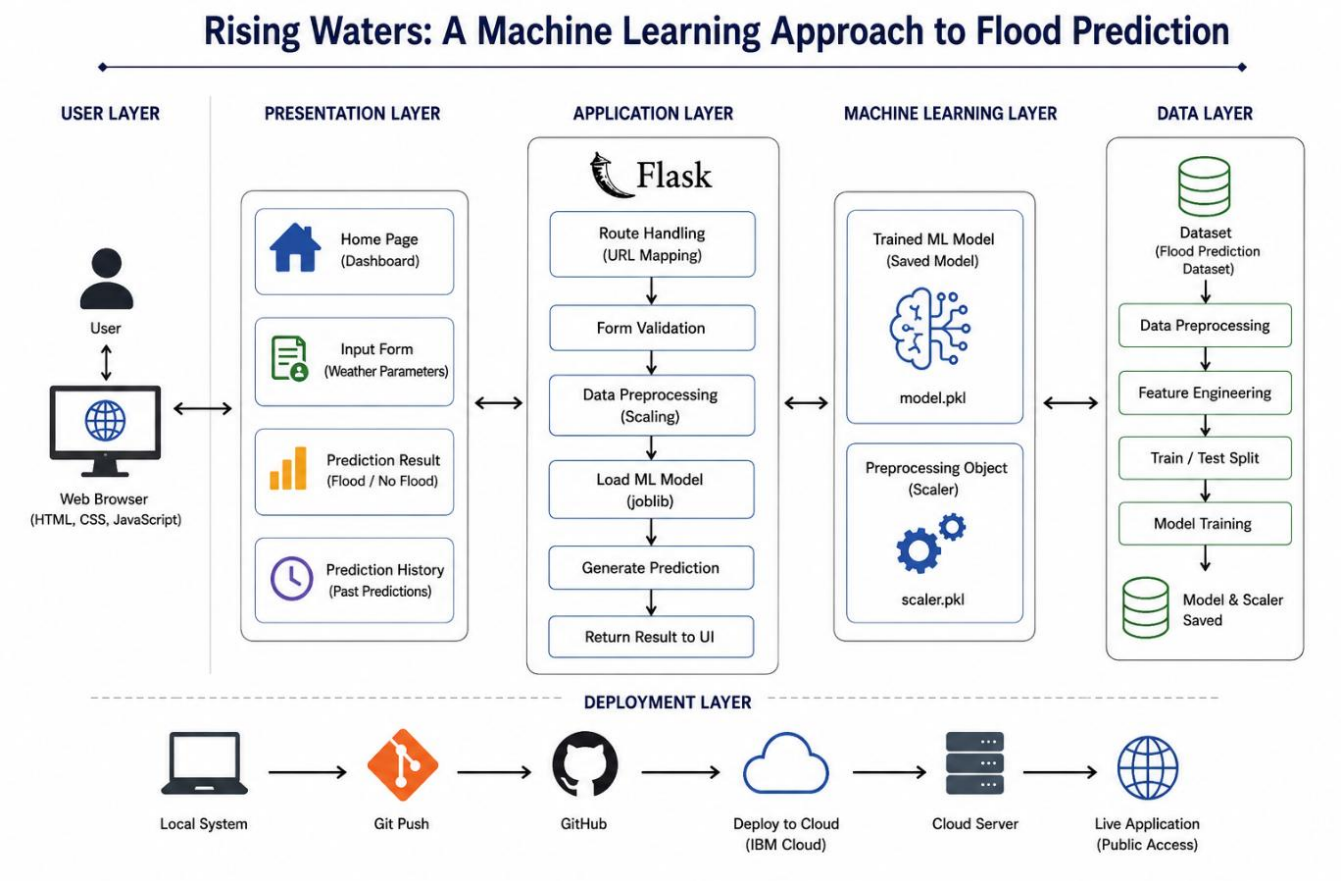
Scenarios

- **Scenario 1: Disaster Management Preparation:** An emergency management official needs to assess the flood risk for an upcoming monsoon season. The official registers an account on the Rising Waters platform and navigates to the prediction dashboard. They manually input the expected seasonal rainfall metrics, along with current temperature, humidity, and cloud cover. The Flask backend processes these inputs, applies the trained XGBoost model, and instantly returns a "HIGH" risk classification along with a specific probability score (e.g., 85%). This rapid, data-driven assessment allows the official to justify early warnings and proactively allocate rescue resources before heavy rainfall begins.

- **Scenario 2: Agricultural Planning:** A local farmer wants to determine if it is safe to plant crops that are highly sensitive to waterlogging. The farmer logs into the system and inputs the historical average rainfall for the region alongside the current humidity and temperature. The application's Machine Learning model evaluates the complex interactions between these variables (such as the Humidity-Rainfall Interaction feature explicitly engineered by the system) and returns a "LOW" risk prediction. Based on this clear risk assessment, the farmer gains the confidence to proceed with the planting schedule without the immediate fear of crop destruction due to flooding.

- **Scenario 3: Historical Tracking and Analysis:** A meteorological research student is studying how varying humidity levels impact flood probabilities when annual rainfall remains constant. The student creates an account and repeatedly uses the prediction form, keeping rainfall inputs static while incrementally changing the humidity percentage. Because the application utilizes a PostgreSQL database linked to the user's session, every single prediction and its exact probability score is saved. The student then navigates to the /history dashboard, where they can view a chronologically ordered, tabular record of all their test cases. This enables them to easily track, compare, and analyze how the model's confidence shifts in response to specific parameter changes over time.

Technical Architecture:



Rising Waters utilizes a secure, modular three-tier architecture:

1. **Frontend (Presentation Layer):** Built with HTML, CSS, and Jinja2 templating. It provides a responsive interface for user registration, data input, and history tracking.
2. **Backend (Application Layer):** A Flask application (app.py) handles all routing, session-based authentication (@login_required), and input sanitization. It acts as the bridge between the user interface, the database, and the Machine Learning models.

3. **Data & Intelligence (Storage & ML Layer):**
    - **PostgreSQL:** Managed via SQLAlchemy ORM, storing User credentials, raw Weather Data inputs, and Prediction Results.
    - **XGBoost & Scikit-Learn:** Pre-trained artifacts (flood_model.pkl, preprocessor.pkl, feature_names.pkl) are loaded into memory to execute predictions on the fly.

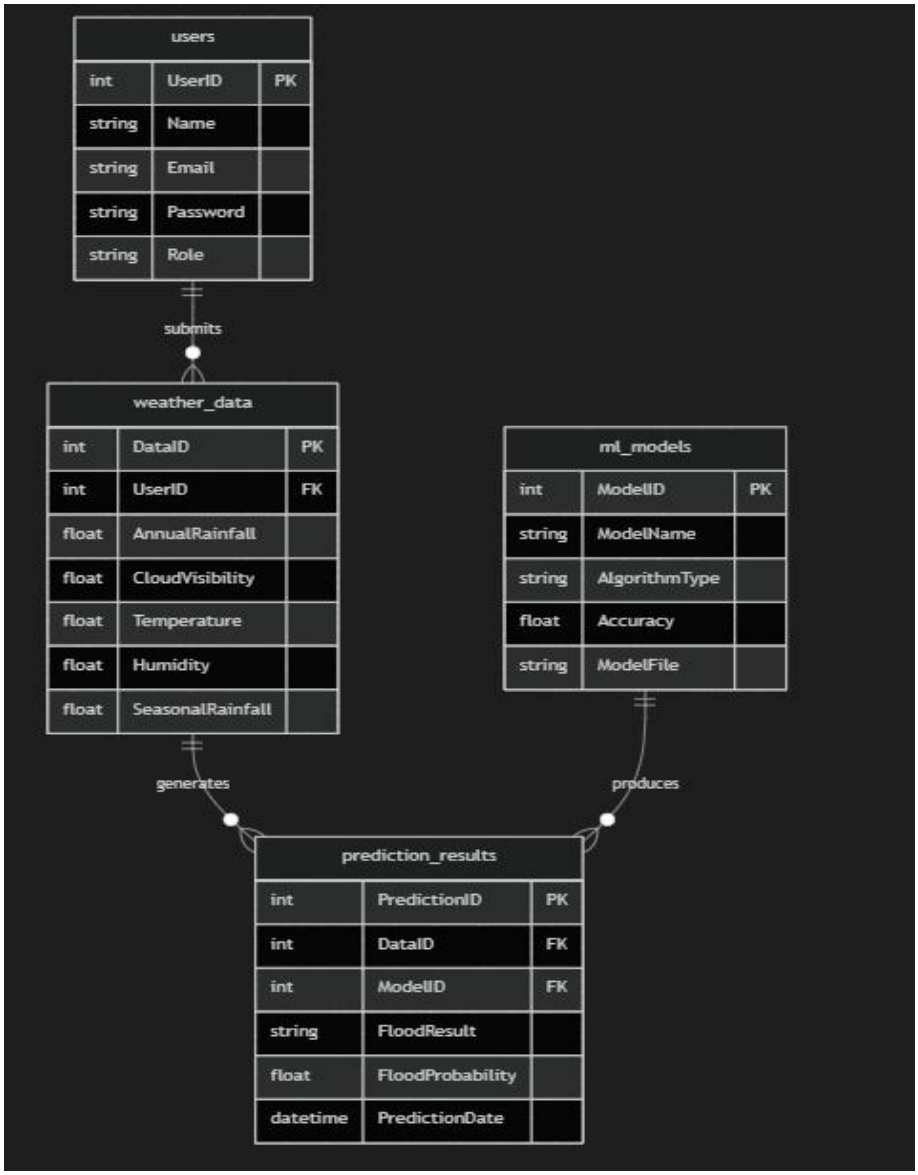
Database Design (Entity-Relationship Diagram)

Rising Waters utilizes PostgreSQL as its primary relational database, orchestrated through the SQLAlchemy Object-Relational Mapper (ORM). This architecture ensures secure data persistence and maintains referential integrity across the system's core entities. The schema is highly normalized into four interconnected tables:

1. **users Table:** This table acts as the foundation of the authentication system. It securely stores user identities, including their Name and Email (which acts as a unique identifier). Passwords are never stored in plaintext; instead, they are cryptographically hashed using werkzeug.security before being saved to the Password column.
   The Role column allows for future expansion into role-based access control (e.g., standard users vs. admins).
2. **ml_models Table:** This table provides an audit trail of the machine learning artifacts used by the system. It tracks the ModelName, the AlgorithmType (e.g., XGBoost), and its evaluated Accuracy. The ModelFile column records the exact file path to the serialized artifact (e.g., flood_model.pkl), ensuring predictions can be traced back to the specific model version that generated them.
3. **weather_data Table:** Whenever an authenticated user submits a prediction request, their raw meteorological inputs (Temperature, Humidity, Cloud Visibility, Annual and Seasonal Rainfall) are recorded here. Crucially, each record includes a foreign key (UserID) linking the data directly back to the user who submitted it, enabling personalized, secure prediction histories.

4. **prediction_results Table:** This table captures the final output of the ML pipeline. It stores the categorical FloodResult ("HIGH" or "LOW") and the exact numeric FloodProbability. To maintain strict relational integrity, this table uses dual foreign keys: DataID links the result directly to the specific weather parameters that triggered it in the weather_data table, and ModelID links it to the model version used. It also records a precise PredictionDate timestamp for chronological sorting in the user dashboard.

Data Flow & Integrity: The database is designed with strict referential constraints. A PredictionResult cannot exist without its parent WeatherData, and WeatherData is bound to a specific User. This structured hierarchy ensures that when an account or prediction is deleted, all related orphaned records are handled properly.



Pre-requisites

To fully understand, run, and modify the Rising Waters application, developers must possess foundational knowledge across the entire web and machine learning stack.

- **Python Programming Proficiency:** Python 3.12.3 is required. Developers must understand core syntax, data structures, and object-oriented programming to navigate app.py and train.py.
- **Flask Framework Knowledge:** Essential for understanding backend routing (@app.route), session management (session), request handling (request.get_json()), and Jinja2 template rendering (render_template).
- **PostgreSQL Database Management:** Familiarity with relational databases, primary/foreign key constraints, and SQL syntax. Knowledge of SQLAlchemy ORM is required to understand how Python classes map to database tables.
- **Machine Learning Fundamentals:** A strong grasp of classification algorithms (specifically decision trees and XGBoost), data scaling (StandardScaler), cross-validation techniques, and metrics like ROC-AUC.
- **HTML, CSS, JavaScript Skills:** Required for modifying the frontend. Understanding of HTML5 form validation, CSS for responsive design, and vanilla JavaScript for handling asynchronous POST requests (AJAX/Fetch API).
- **Environment Management:** Familiarity with python-dotenv for securely loading credentials from a .env file without hardcoding them into the application script.

Project Workflow

Milestone 1: Project Planning and Environment Setup

In this milestone, we focus on configuring the foundation of the Rising Waters application. This involves preparing the Python environment, setting up the

PostgreSQL database connection, and defining the system architecture before writing the core logic.

Activity 1.1: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.12.3 is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```bash
python -m venv venv
venv\Scripts\activate
```

- **Install Dependencies:** Install all required libraries including Flask, SQLAlchemy, XGBoost, and Scikit-Learn:

```bash
pip install -r requirements.txt
```

Activity 1.2: Database and Security Configuration Before the application can start securely, you must configure environment variables to handle database connections and session encryption.

- **Create a .env File:** Create a .env file in the root of your project directory and configure your PostgreSQL URI and secret key:

```ini
FLASK_SECRET_KEY=your_secure_random_string
SQLALCHEMY_DATABASE_URI=postgresql://username:password@localhost:5432/rising_waters
```

- **Verify the Configuration is Loaded:** The app.py file loads these variables automatically at startup using python-dotenv:

```python
from dotenv import load_dotenv
load_dotenv()
secret_key = os.environ.get('FLASK_SECRET_KEY')
```

Important — Never Commit Secrets: Add .env to your .gitignore file to prevent accidentally pushing database credentials to a public repository.

Activity 1.3: Define the Architecture of the Application

- **Detail Frontend Functionality:** Outline how users interact with the application by entering meteorological data on the /Predict page and viewing past results on the /history dashboard.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests, securely authenticates users via werkzeug.security, interacts with PostgreSQL via SQLAlchemy, and queries the XGBoost model.

Milestone 2: Data Preprocessing & Model Development

This milestone focuses entirely on the Machine Learning pipeline implemented in train.py. The objective is to load historical flood data, clean it, engineer robust features, and train an accurate classification model.

Activity 2.1: Data Loading and Cleaning

- **Load the Dataset:** Use pandas to ingest flood_dataset_expanded.csv.
- **Handle Missing Values:** Implement the handle_missing_values() function to intelligently impute missing data (using the mean for normal distributions and the median for highly skewed columns).
- **Cap Outliers:** Execute the cap_outliers_iqr() function to prevent extreme weather anomalies from skewing the model weights using the Interquartile Range method.

Activity 2.2: Feature Engineering and Selection

- **Create Derived Features:** Combine existing parameters to create physically meaningful metrics,

  specifically Total_Seasonal_Rainfall and Humidity_Rainfall_Interaction.
- **Prevent Multicollinearity:** Calculate the correlation matrix and drop features with a correlation > 0.90, keeping only the one most strongly correlated with the target variable.

- **Handle Class Imbalance:** Apply SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic flood events, ensuring the model doesn't overfit to the majority "No Flood" class.

Activity 2.3: Model Training and Serialization

- **Evaluate Algorithms:** Use 10-fold Stratified Cross-Validation to evaluate Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost across metrics like Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- **Serialize Artifacts:** Select the best-performing model (XGBoost), fit a StandardScaler, and save the artifacts alongside the ordered feature names into the models/ directory using joblib:

```python
joblib.dump(model, os.path.join(MODELS_DIR, 'flood_model.pkl'))
joblib.dump(scaler, os.path.join(MODELS_DIR, 'preprocessor.pkl'))
```

Milestone 3: App.py Development

Milestone 3 focuses on creating the core logic of the app.py file, serving as the backbone of the Rising Waters application. You will set up database connections, authentication routes, and the ML inference pipeline.

Activity 3.1: Database Initialization and Artifact Loading

- **Load ML Artifacts:** Implement load_models() to globally load the .pkl files generated in Milestone 2. If these files are missing, the app raises a FileNotFoundError.
- **Test Database Connection:** Execute a raw SQL query to verify PostgreSQL health before mapping tables:

python

db.session.execute(db.text('SELECT 1'))

db.create_all()

Activity 3.2: Defining Core Authentication Routes

- **Establish Public Routes:** Create the /login and /register endpoints to handle user onboarding. Hash incoming passwords before database insertion:

python

hashed_password = generate_password_hash(password, method='pbkdf2:sha256')

- **Protect Private Routes:** Implement a custom @login_required decorator to protect the prediction and history endpoints, redirecting unauthenticated users to the login page.

Activity 3.3: Implement the Prediction Endpoint (/predict)

- **Process User Input:** Extract the JSON payload using request.get_json() and validate that critical parameters like temp and humidity fall within realistic meteorological boundaries.
- **Apply Inference Pipeline:** Re-apply the exact feature engineering logic (Total_Seasonal_Rainfall) used in train.py, scale the input via preprocessor.transform(), and execute model.predict_proba().
- **Save and Cache:** Save the result to the PostgreSQL prediction_results table, linked to the user's ID, and invalidate the user's specific history cache (history_user_{user_id}) to ensure data consistency.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface using Jinja2 templates, HTML, and CSS.

Activity 4.1: Designing the User Interface

- **Set Up Base HTML Structure:** Develop standard layouts with navigation bars connecting Home, Predict, History, and Logout routes.
- **Design Responsive Layouts:** Utilize CSS (e.g., main.css) to ensure the input forms and data tables render cleanly on both desktop and mobile devices.

Activity 4.2: Creating Dynamic Content Rendering

- **Build the Prediction Form:** Create inputs for all 8 weather parameters with client-side boundary validation (min/max attributes).
- **Render History Dynamically:** Use Flask's render_template to pass database query results to the frontend. Utilize Jinja2 {% for res in history %} loops to dynamically construct a tabular view of the user's past predictions.

Milestone 5: Testing and Verification

In Milestone 5, the focus is on verifying correct operation locally and ensuring the system can handle concurrent user load.

Activity 5.1: Local Testing and Verification

- **Start the Flask Development Server:**

  bash

  python app.py
- **Interact with the Application:** Open a browser and navigate to http://127.0.0.1:5000. Register a new test user, log in, submit a high-risk weather profile, and verify that the database correctly logs the prediction.

Activity 5.2: Performance Load Testing

- **Run Locust:** Execute the integrated locustfile.py to benchmark the performance of the system under stress.

  bash

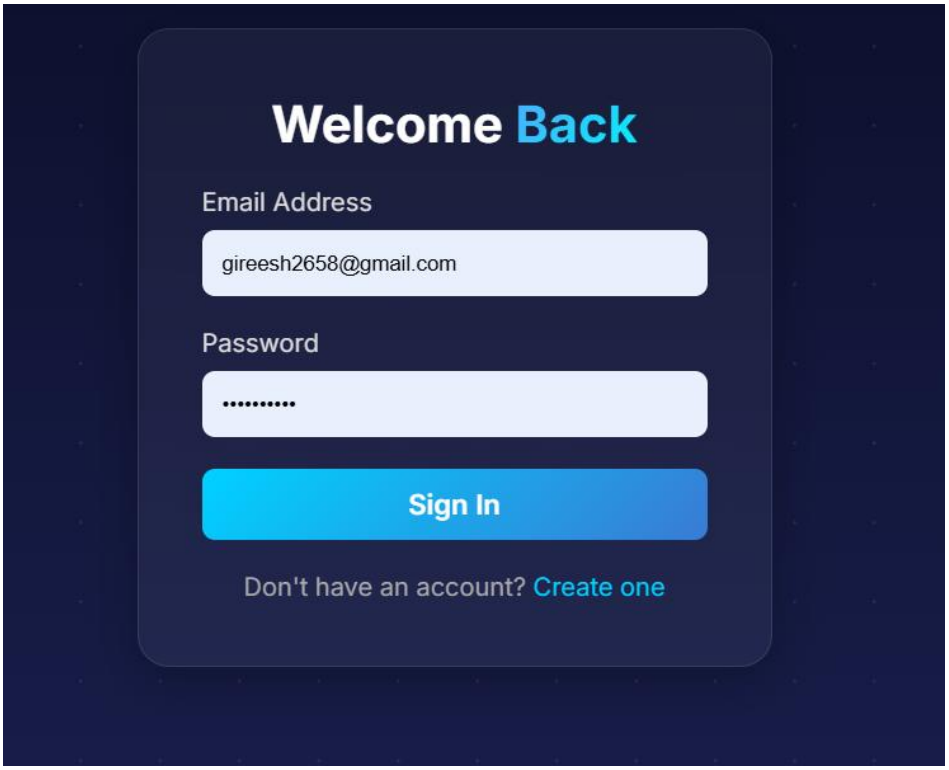
  locust -f locustfile.py
- **Verify Throughput:** Analyze the response times of the / and heavily cached /history endpoints to ensure the application remains stable during traffic spikes.

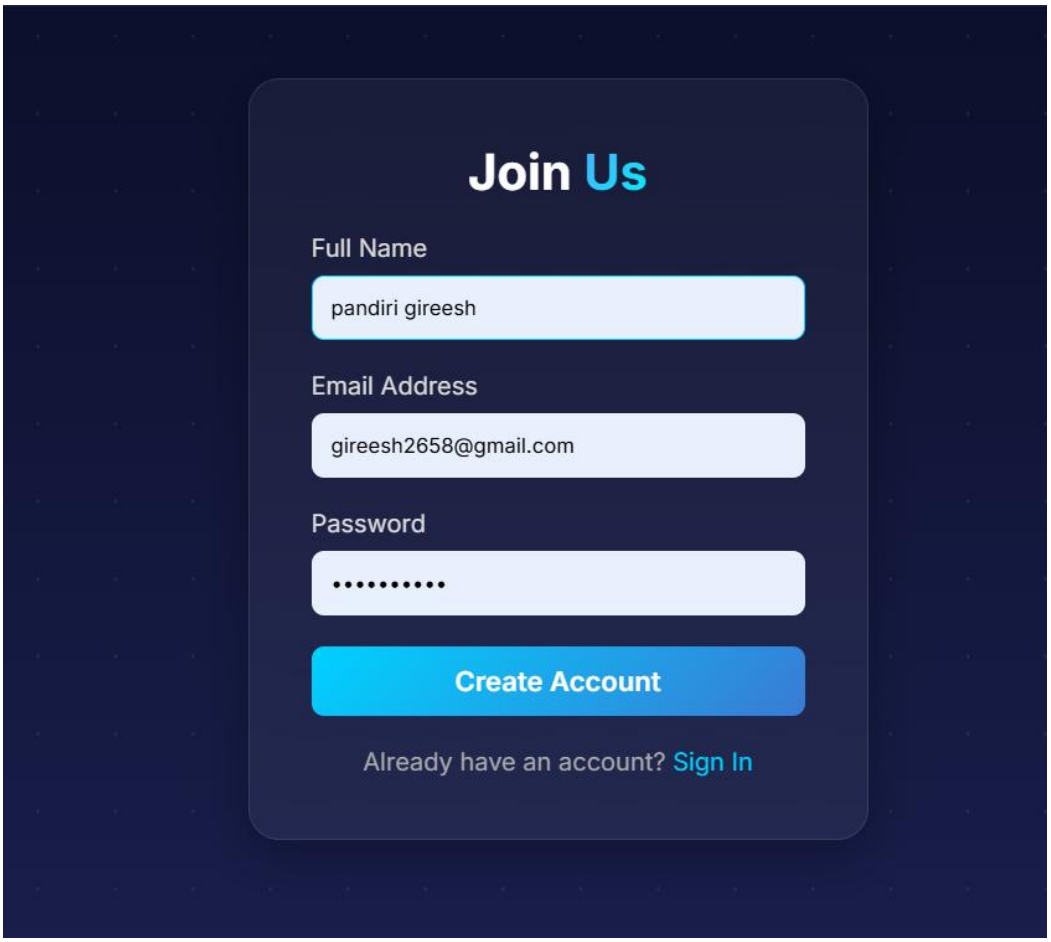
Milestone 6: Conclusion

Rising Waters demonstrates a highly robust, fully integrated machine learning application. By combining advanced data preprocessing with a secure, database-backed web framework (PostgreSQL, Flask Auth), the project successfully transitions a predictive algorithm into a production-ready, user-centric disaster management tool.

Exploring the Web Pages

The frontend of Rising Waters is designed to be highly responsive and user-centric. All pages utilize Jinja2 template inheritance (e.g., {% extends 'base.html' %}), ensuring a consistent navigation bar and unified CSS styling (Grid/Flexbox layouts, hover states, and smooth transitions) across the entire application.

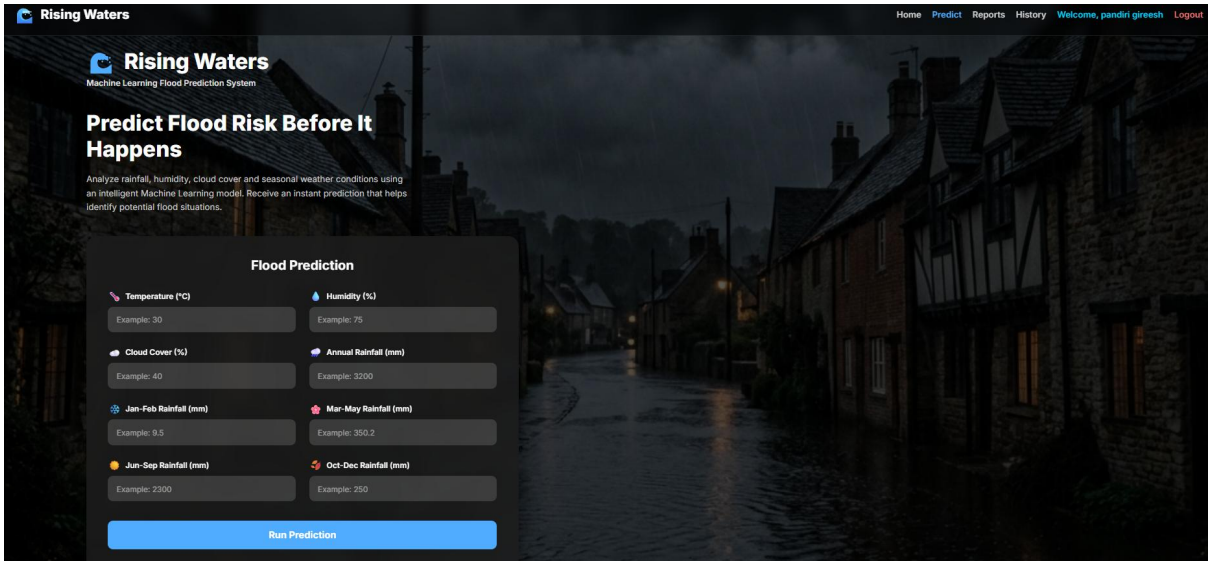

Welcome Back
Email Address
gireesh2658@gmail.com
Password
••••••••••
Sign In
Don't have an account? Create one


Join Us
Full Name
pandiri gireesh
Email Address
gireesh2658@gmail.com
Password
••••••••••
Create Account
Already have an account? Sign In

Authentication Pages (Login / Register)

Description: Dedicated interfaces for secure user onboarding, designed with clean, distraction-free card layouts.

- **Registration Flow:** Users provide a Name, Email, and Password. The HTML form uses strict required attributes for client-side validation, ensuring no empty submissions reach the server. Upon submission, the Flask backend queries the PostgreSQL database to enforce email uniqueness. If unique, the password is cryptographically hashed (werkzeug.security) and securely inserted into the users table.
- **Login Flow:** Users submit their credentials via a streamlined interface. The backend uses check_password_hash to securely verify the identity. Upon success, a secure Flask session is established (session['user_id'] = user.UserID), granting access to protected dashboards. Flask's flash() messages dynamically inject Bootstrap-style alerts (e.g., "success" or "danger" banners) to provide instant visual feedback on authentication attempts.



Prediction Dashboard (/Predict)

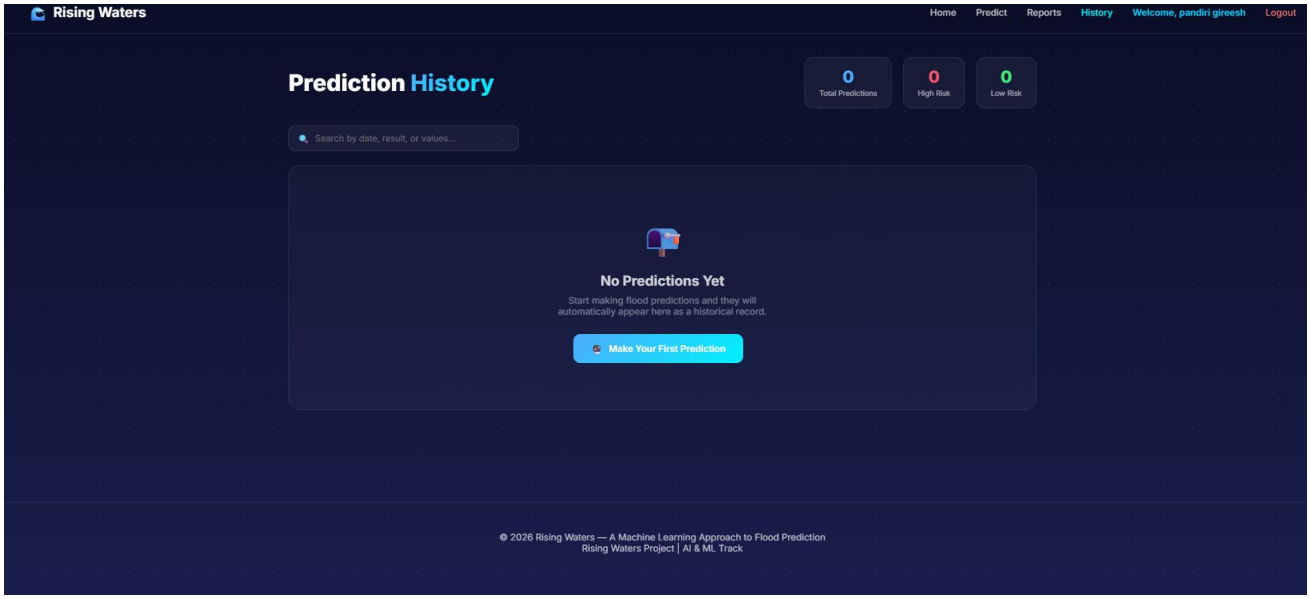
Description: The core interactive workspace for meteorological data entry, featuring a multi-column responsive grid layout that adapts flawlessly from desktop to mobile screens.

- **User Input:** The form requires 8 specific numerical parameters organized logically: Temperature, Humidity, Cloud Cover, Annual Rainfall, and the four seasonal rainfall blocks (Jan-Feb, Mar-May, Jun-Sep, Oct-Dec).
- **Validation:** Client-side HTML boundaries (e.g., min="0" max="100" for humidity) prevent impossible weather scenarios (like negative rainfall or 200% humidity) from being submitted, saving server bandwidth and preventing ML model corruption.
- **Submission:** The form submission is designed to be seamless. Upon clicking the "Predict Risk" button, the data is packaged and sent via an asynchronous POST request to the backend API (/predict), bypassing clunky full-page reloads and providing a modern, Single-Page Application (SPA) feel.

Results Display Layer

Description: The dynamic feedback layer that renders seamlessly below or alongside the prediction form immediately after the API returns the result.

- **Data Processing:** The Flask backend instantly processes the JSON payload, applies the preprocessor.pkl to scale the inputs, executes the model.predict_proba() function on the XGBoost artifact, and packages the result.
- **Visual Output:** The UI conditionally alters its CSS classes based on the JSON response. If the risk is "HIGH", the DOM dynamically renders prominent alert styling (often utilizing stark red color palettes and warning icons) alongside the exact probability score (e.g., "85.2%"). It displays a critical advisory message: "Immediate precautionary measures recommended." If "LOW", the UI transitions to safe, reassuring green visual cues, signaling that conditions are normal.



Prediction History Dashboard (/history)

Description: A personalized, persistent analytics dashboard allowing users to track and evaluate their historical risk assessments over time.

- **Backend Query:** When the route is accessed, SQLAlchemy executes a highly structured JOIN query (PredictionResult.query.join(WeatherData).filter(...)) to retrieve all past predictions tied specifically to the authenticated user's active session['user_id'].
- **Performance Optimization:** To prevent database bottlenecks when querying large histories, this specific view is heavily optimized using Flask-Caching. The data is retrieved directly from high-speed memory using a custom user_cache_key().
- **Tabular View:** Jinja2 templating iterates over the result set ({% for res in history %}), rendering a clean, chronological HTML table. This table displays the exact timestamp, the raw input parameters (Temperature, Rainfall, etc.), and the resulting ML classification, empowering users to perform long-term meteorological trend analysis.